KEYPOSTURE

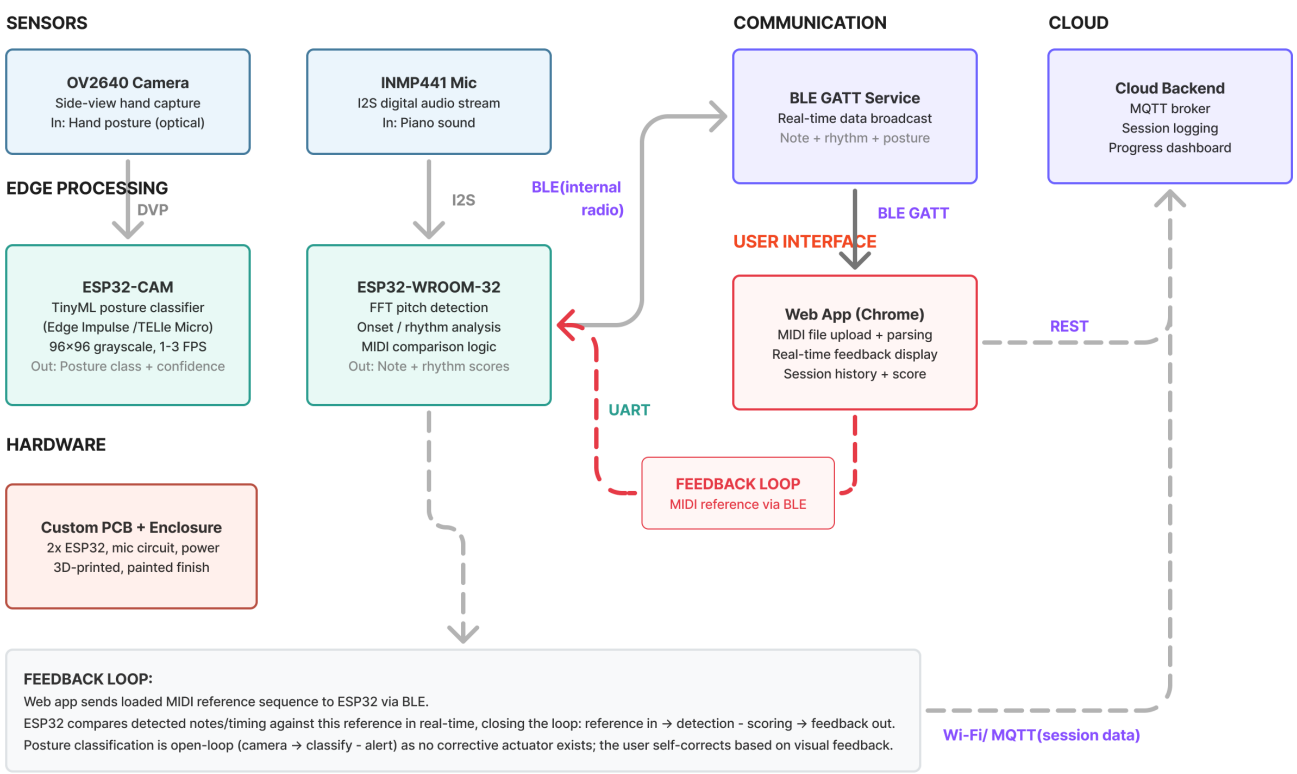
Project Register & Ownership | $350 Budget

Section 1: System Mission & Boundary

This system is a smart piano posture correction product that provides real-time feedback on hand posture and basic playing accuracy during practice. It is designed for beginner piano learners who need immediate, actionable guidance without a human instructor.

The system uses an OV2640 camera with ESP32-CAM for on-device posture classification, an INMP441 microphone with ESP32 for basic audio analysis, BLE communication to a web app for feedback visualisation, and optional cloud storage for session tracking.

Section 2: Functional Architecture Diagram



Section 3: Risk Reduction Prototypes

#	Name + Owner	Assumption Being Validated	Inputs → Process → Outputs	Success Criteria
1	**Rhythm Onset Scorer** Yuwen Chen	Spectral-flux onset detection can timestamp note attacks accurately enough to score rhythm against a MIDI reference with meaningful resolution.	In: 30 note sequences played to a metronome → Process: Spectral flux onset detection + comparison to MIDI timing → Out: Per-note timing error (ms)	Onset timing error ≤30ms vs ground truth for ≥80% of notes. Early/late/on-beat classification is correct ≥85% of the time.
2	**Posture Signal Viability** Yunxiao Du	A side-view ESP32-CAM at 96×96 grayscale captures enough visual variance to distinguish correct hand posture from 2–3 common bad postures.	In: 100+ images per class → Process: Visual inspection + histogram analysis of inter-class variance → Out: Confirmed class separability	Clear visual differences across classes confirmed. Pixel intensity distributions show statistically significant separation ($p < 0.05$).
3	**On-Device Posture Classifier** Davi Dai	A quantised CNN trained via Edge Impulse can classify hand posture on ESP32-CAM at useful accuracy and frame rate for real-time feedback.	In: 150+ labelled images per class → Process: Train CNN in Edge Impulse, deploy to ESP32-CAM → Out: Posture class at N FPS	≥80% validation accuracy on 3-class problem. Inference runs at ≥1.5 FPS on ESP32-CAM. Model size < 250KB.
4	**MIDI Sync Prototype** Sijia Shao	A MIDI file can be parsed in-browser and its note sequence synchronised with incoming BLE note events to produce a meaningful correctness score.	In: Standard MIDI file + simulated BLE note stream → Process: JS MIDI parser (Tone.js) + real-time matching algorithm → Out: Per-note correct/incorrect	MIDI parser correctly extracts note sequence from 3 test files. Matching algorithm scores ≥90% of notes correctly against simulated input.